



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 04

NOMBRE COMPLETO: Rosas Cañada Abraham

N° de Cuenta: 318307866

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 15/03/2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1.- Terminar la Grúa con:

Base (pirámide cuadrangular) — la jerarquía establece la cabina como nodo padre de la base, y la base como nodo padre de las 4 llantas. Cada llanta hereda las transformaciones acumuladas a través de la matriz auxiliar modelBase.

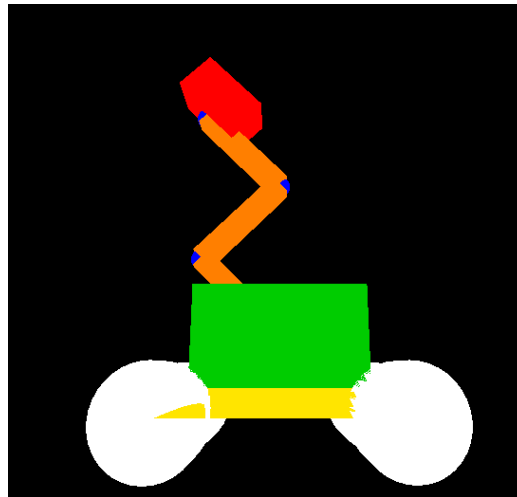


Figura 1. Vista general de la grúa con base piramidal y 4 llantas.

```
//base - nodo hijo de la cabina, hereda de modelaux (pivot de la cabina)
//la punta de la pirámide (Y=+0.5 local) queda en el origen de la cabina (0,7,-4)
//translate -1.5 en Y baja el centro para que la punta coincida con ese origen
//escala (9,3,7): 2 unidades más ancha que la cabina en X y Z
//modelBase guarda el pivot de la base sin escala - nodo padre de las llantas
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
modelBase = model;
model = glm::scale(model, glm::vec3(9.0f, 3.0f, 7.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 0.9f, 0.0f); // amarillo
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMeshGeometry();
```

Figura 2. Bloque de código — jerarquía cabina → base → llantas.

4 llantas (4 cilindros) — con teclas V, B, N, M se puede girar cada llanta individualmente. Cada cilindro hereda la posición de su esquina correspondiente de la base a través de `modelBase`.

```
//llanta frente derecha - articulacion5 tecla V
//pivot en esquina frente derecha de la base (+4.5, -1.5, +3.5) relativo a modelBase
model = modelBase;
model = glm::translate(model, glm::vec3(4.5f, -1.5f, 3.5f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f); // blanco
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();
```

Figura 3. Código de la llanta frente derecha (articulacion5, tecla V).

```
//llanta frente izquierda - articulacion6 tecla B
//pivot en esquina frente izquierda de la base (-4.5, -1.5, +3.5) relativo a modelBase
model = modelBase;
model = glm::translate(model, glm::vec3(-4.5f, -1.5f, 3.5f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();
```

Figura 4. Código de la llanta frente izquierda (articulacion6, tecla B).

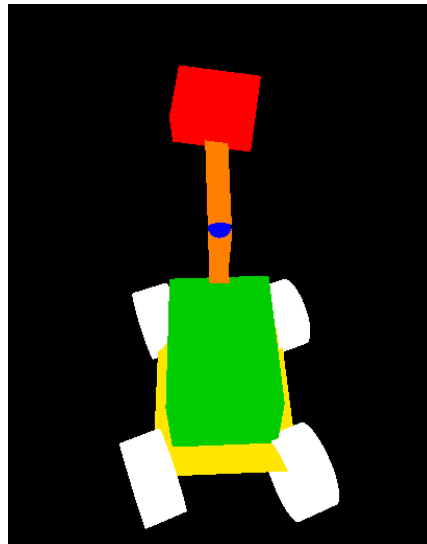


Figura 5. Ejecución — grúa completa con base y llantas visibles.

```
//llanta atrás izquierda - articulacion8 tecla M
//pivot en esquina atrás izquierda de la base (-4.5, -1.5, -3.5) relativo a modelBase
model = modelBase;
model = glm::translate(model, glm::vec3(-4.5f, -1.5f, -3.5f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();
```

Figura 6. Código de la llanta trasera derecha (articulacion7, tecla N).

```

//llanta atrás derecha - articulacion7 tecla N
//pivot en esquina atrás derecha de la base (+4.5, -1.5, -3.5) relativo a modelBase
model = modelBase;
model = glm::translate(model, glm::vec3(4.5f, -1.5f, -3.5f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();

```

Figura 7. Código de la llanta trasera izquierda (articulacion8, tecla M).

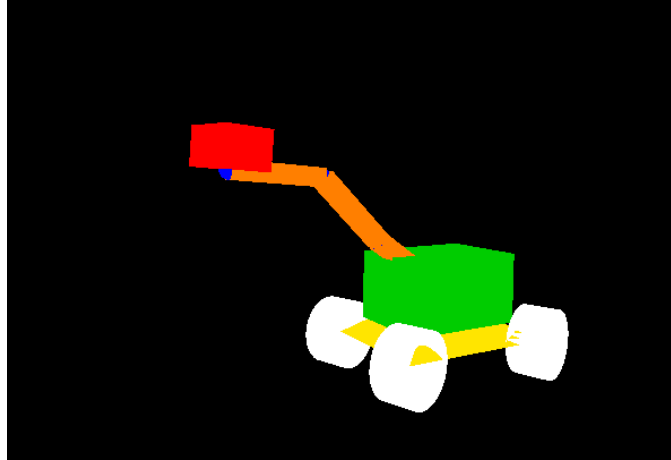


Figura 8. Resultado final de la grúa con llantas articuladas.

2.- Crear un animal robot 3D

Se implementó un perro robot articulado usando modelado jerárquico. El torso actúa como nodo raíz del que heredan: cuello → cabeza → orejas (conos), cola articulada en 2 segmentos, y 4 patas con 2 articulaciones cada una (cadera amarilla + muslo + rodilla + canilla).

```
void CrearCubo()
{
    unsigned int cubo_indices[] = {
        0,1,2, 2,3,0, 1,5,6, 6,2,1,
        7,6,5, 5,4,7, 4,0,3, 3,7,4,
        4,5,1, 1,0,4, 3,2,6, 6,7,3
    };
    GLfloat cubo_vertices[] = {
        -0.5f,-0.5f, 0.5f,  0.5f,-0.5f, 0.5f,
        0.5f, 0.5f, 0.5f, -0.5f, 0.5f, 0.5f,
        -0.5f,-0.5f,-0.5f, 0.5f,-0.5f,-0.5f,
        0.5f, 0.5f,-0.5f, -0.5f, 0.5f,-0.5f
    };
    Mesh* cubo = new Mesh();
    cubo->CreateMesh(cubo_vertices, cubo_indices, 24, 36);
    meshList.push_back(cubo); // indice 0
}
```

Figura 9. Vista lateral del perro robot articulado.

```
void CrearCilindro(int res, float R)
{
    //cilindro con eje en Y, tapas cerradas
    int n, i;
    GLfloat dt = 2 * PI / res, x, z;
    vector<GLfloat> vertices;
    vector<unsigned int> indices;

    // cuerpo lateral
    for (n = 0; n <= res; n++) {
        x = (n != res) ? R * cos(n * dt) : R;
        z = (n != res) ? R * sin(n * dt) : 0.0f;
        vertices.push_back(x); vertices.push_back(-0.5f); vertices.push_back(z);
        vertices.push_back(x); vertices.push_back(0.5f); vertices.push_back(z);
    }

    // tapa inferior: centro + abanico
    vertices.push_back(0.0f); vertices.push_back(-0.5f); vertices.push_back(0.0f);
    for (n = 0; n <= res; n++) {
        x = R * cos(n * dt); z = R * sin(n * dt);
        vertices.push_back(x); vertices.push_back(-0.5f); vertices.push_back(z);
    }

    // tapa superior: centro + abanico
    vertices.push_back(0.0f); vertices.push_back(0.5f); vertices.push_back(0.0f);
    for (n = 0; n <= res; n++) {
        x = R * cos(n * dt); z = R * sin(n * dt);
        vertices.push_back(x); vertices.push_back(0.5f); vertices.push_back(z);
    }

    for (i = 0; i < (int)vertices.size(); i++) indices.push_back(i);
    Mesh* cilindro = new Mesh();
    cilindro->CreateMeshGeometry(vertices, indices, vertices.size(), indices.size());
    meshList.push_back(cilindro); // indice 1
}
```

Figura 10. Vista frontal mostrando orejas y patas.

Instanciando cubos, cilindros, conos y esferas:

Se crean tres primitivas geométricas indexadas en meshList, más una esfera global de la clase Sphere, que se utiliza para cabeza, rodillas y articulaciones de la cola.

```
// Creación e instanciación de primitivas geométricas
CrearCubo();           // índice 0 – cubo unitario centrado en origen
CrearCilindro(36, 1.0f); // índice 1 – cilindro cerrado con tapas, eje en Y
CrearCono(36, 1.0f);    // índice 2 – cono con tapa base, punta en Y=+0.5

// Esfera – instancia global, radio=1, 20 segmentos lat/lon
Sphere sp = Sphere(1.0, 20, 20);
sp.init();
sp.load();
// Uso dentro del loop de renderizado:
sp.render();           // dibuja la esfera con la matriz model activa
```

```
void CrearCono(int res, float R)
{
    //cono con punta en Y=+0.5 y base circular en Y=-0.5, tapa base cerrada
    int n;
    GLfloat dt = 2 * PI / res, x, z;
    vector<GLfloat> vertices;
    vector<unsigned int> indices;

    // punta
    vertices.push_back(0.0f); vertices.push_back(0.5f); vertices.push_back(0.0f);
    // costado
    for (n = 0; n <= res; n++) {
        x = R * cos(n * dt); z = R * sin(n * dt);
        vertices.push_back(x); vertices.push_back(-0.5f); vertices.push_back(z);
    }
    // tapa base: centro + abanico
    vertices.push_back(0.0f); vertices.push_back(-0.5f); vertices.push_back(0.0f);
    for (n = 0; n <= res; n++) {
        x = R * cos(n * dt); z = R * sin(n * dt);
        vertices.push_back(x); vertices.push_back(-0.5f); vertices.push_back(z);
    }

    for (n = 0; n < (int)vertices.size(); n++) indices.push_back(n);
    Mesh* cono = new Mesh();
    cono->CreateMeshGeometry(vertices, indices, vertices.size(), indices.size());
    meshList.push_back(cono); // índice 2
}
```

Figura 11. Fragmento de código — funciones CrearCubo, CrearCilindro y CrearCono.

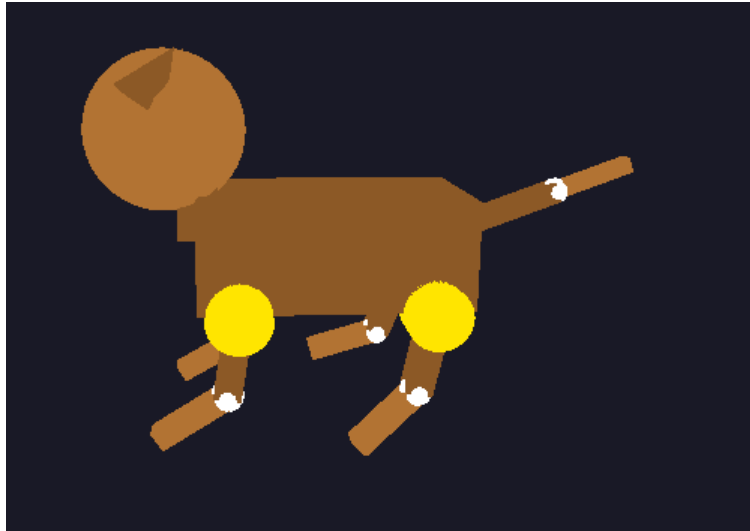


Figura 12. Ejecución mostrando torso, cuello y cabeza instanciados.

```
Sphere sp = Sphere(1.0f, 20, 20);
```

Figura 13. Líneas de instanciación en main().

4 patas articuladas en 2 partes:

Cada pata sigue la cadena: `modelTorso` → cadera (cilindro amarillo, fijo) → muslo (cubo café, articulación en Z) → esfera rodilla (blanca) → canilla (cubo café claro, articulación en Z). El pivot de cada cadera se guarda en `modelCaderaXX` antes del `scale` para no heredar la escala a los hijos.

```
// — PATA DELANTERA DERECHA — Z=muslo, X=canilla —————
// cilindro de cadera (amarillo) — nodo hijo del torso, fijo
model = modelTorso;
model = glm::translate(model, glm::vec3(3.5f, -2.0f, 3.0f));
modelCaderaDD = model; // pivot de cadera, padre del muslo
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.2f, 1.5f, 1.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(amarillo));
meshList[1]->RenderMeshGeometry();

// muslo — articulacion5 (tecla Z), rota en Z desde el pivot de cadera
model = modelCaderaDD;
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f,
0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
modelaux = model;
```

```

model = glm::scale(model, glm::vec3(1.0f, 3.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(cafe));
meshList[0]->RenderMesh();

// esfera rodilla – blanca, pivot de canilla
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f,
0.0f, 1.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(blanco));
sp.render();

// canilla – articulacion6 (tecla X), cuelga de la rodilla
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.8f, 3.0f, 0.8f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(cafeClaro));
meshList[0]->RenderMesh();

// — PATA DELANTERA IZQUIERDA – C=muslo, V=canilla —————
model = modelTorso;
model = glm::translate(model, glm::vec3(3.5f, -2.0f, -3.0f));
modelCaderaDI = model;
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(150.0f), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.2f, 1.5f, 1.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(amarillo));
meshList[1]->RenderMeshGeometry();

model = modelCaderaDI;
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(0.0f,
0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.0f, 3.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(cafe));
meshList[0]->RenderMesh();

model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f,
0.0f, 1.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(blanco));
sp.render();

model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.8f, 3.0f, 0.8f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));

```



```

glUniform3fv(uniformColor, 1, glm::value_ptr(cafeClaro));
meshList[0]->RenderMesh();

// — PATA TRASERA DERECHA — B=muslo, N=canilla —————
model = modelTorso;
model = glm::translate(model, glm::vec3(-3.5f, -2.0f, 3.0f));
modelCaderaTD = model;
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.2f, 1.5f, 1.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(amarillo));
meshList[1]->RenderMeshGeometry();

model = modelCaderaTD;
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion9()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.0f, 3.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(cafe));
meshList[0]->RenderMesh();

model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion10()), glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(blanco));
sp.render();

model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.8f, 3.0f, 0.8f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(cafeClaro));
meshList[0]->RenderMesh();

// — PATA TRASERA IZQUIERDA — M=muslo, N=canilla —————
model = modelTorso;
model = glm::translate(model, glm::vec3(-3.5f, -2.0f, -3.0f));
modelCaderaTI = model;
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(150.0f), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.2f, 1.5f, 1.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(amarillo));
meshList[1]->RenderMeshGeometry();

model = modelCaderaTI;
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion11()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.0f, 3.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(cafe));

```

```

meshList[0]->RenderMesh();

model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion12()),
glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(cafeclaro));
sp.render();

model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.8f, 3.0f, 0.8f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(cafeclaro));
meshList[0]->RenderMesh();

```



Figura 14. Patas delanteras articuladas — teclas Z/X y C/V.



Figura 15. Patas traseras articuladas — teclas B/N y M/.

Cola articulada y 2 orejas articuladas:

Las orejas son conos hijos de modelCabeza, con rotación fija de 35° en Z para la inclinación natural y articulación en X con teclas F y G respectivamente. La cola parte del extremo trasero del torso con -20° inicial (posición feliz), se articula en Z con tecla K en el primer segmento y tecla L en el segundo.

```
// OREJA IZQUIERDA – hija de la cabeza, cono, articulacion1 (F)
model = modelCabeza;
model = glm::translate(model, glm::vec3(0.0f, 2.1f, 2.2f));
model = glm::rotate(model, glm::radians(35.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f, 2.2f, 0.7f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(cafe));
meshList[2]->RenderMeshGeometry();

// OREJA DERECHA – simétrica, articulacion2 (G)
model = modelCabeza;
model = glm::translate(model, glm::vec3(0.0f, 2.1f, -2.2f));
model = glm::rotate(model, glm::radians(35.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f, 2.2f, 0.7f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(cafe));
meshList[2]->RenderMeshGeometry();

// COLA – segmento 1, articulacion3 (K), posición inicial -20° (arriba)
model = modelTorso;
model = glm::translate(model, glm::vec3(-5.0f, 0.5f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(-20.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(-2.0f, 0.0f, 0.0f));
modelCola = model;
model = glm::scale(model, glm::vec3(4.0f, 0.8f, 0.8f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(cafe));
meshList[0]->RenderMesh();

// esfera articulación cola – blanca
model = modelCola;
model = glm::translate(model, glm::vec3(-2.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr( blanco ));
sp.render();

// COLA – segmento 2, articulacion4 (L)
model = modelaux;
model = glm::translate(model, glm::vec3(-1.5f, 0.0f, 0.0f));
```

```
model = glm::scale(model, glm::vec3(3.0f, 0.6f, 0.6f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
glUniform3fv(uniformColor, 1, glm::value_ptr(cafecClaro));  
meshList[0]->RenderMesh();
```

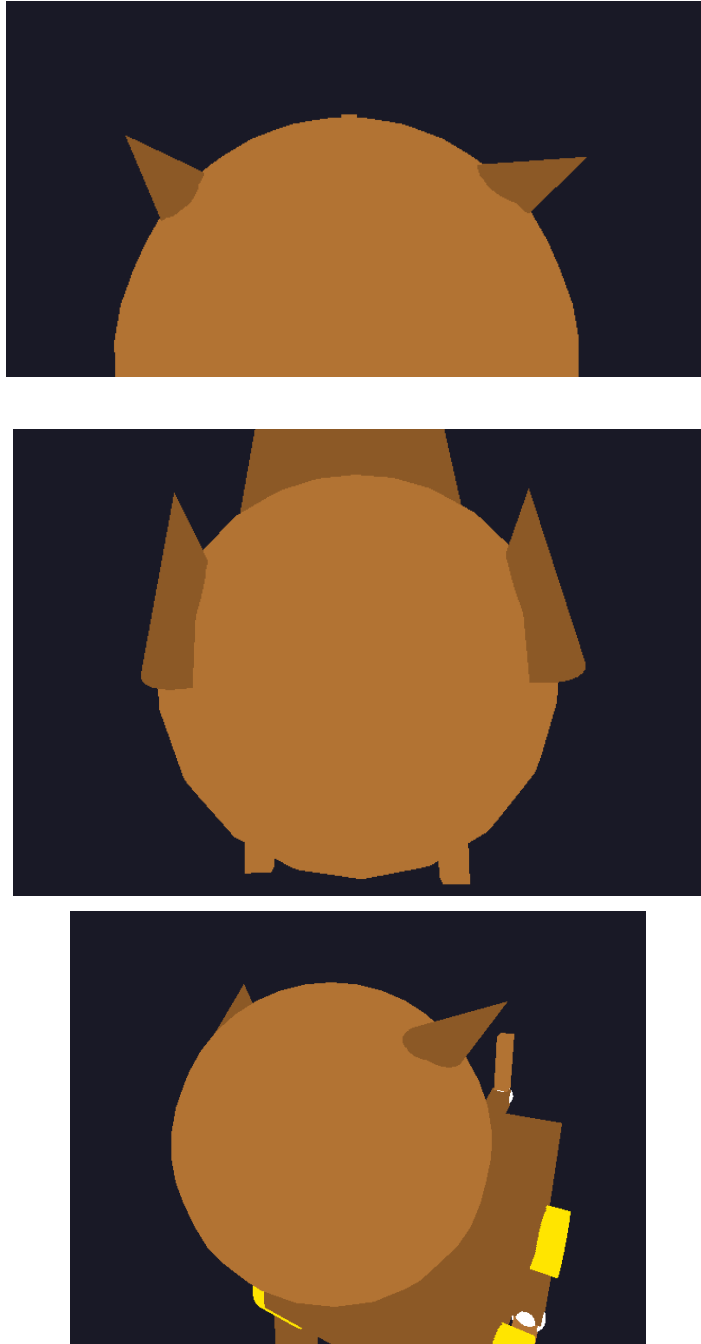


Figura 16. Orejas articuladas con conos — teclas F y G.

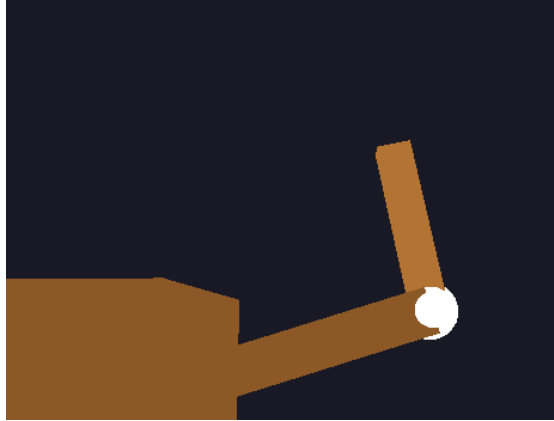


Figura 17. Cola en posición inicial — teclas K y L.

2.- Problemas encontrados

Durante el desarrollo de la práctica surgieron varios inconvenientes. El primero fue la orientación incorrecta de los cilindros de las llantas: al rotar 90° en X, el cilindro quedaba con las tapas huecas visibles desde el frente; se corrigió agregando abanico de vértices en las tapas inferior y superior dentro de `CrearCilindro`. El segundo problema fue la herencia incorrecta de escala en la jerarquía: al guardar `modelaux` después del `glm::scale`, los nodos hijos heredaban las escalas acumuladas y se deformaban; la solución fue siempre guardar el pivot antes del `scale`. Un tercer problema fue la orientación de las orejas (conos): la punta apuntaba en dirección equivocada; se resolvió combinando rotaciones fijas de -90° en X y 180° en Y para obtener la posición natural. Finalmente, la cola inicializaba apuntando hacia abajo; se corrigió cambiando el ángulo inicial de 120° a -20° en Z.

3.- Conclusión

La práctica de modelado jerárquico resultó de complejidad media-alta, principalmente por la necesidad de comprender con precisión el orden de las transformaciones en la pila de matrices y la correcta gestión de los pivotes mediante `modelaux`. El uso de `glm::translate`, `glm::rotate` y `glm::scale` en cadena exige cuidado al guardar el estado intermedio antes del `scale` para no propagar deformaciones a los nodos hijos. La práctica estuvo bien estructurada aunque se habría beneficiado de una explicación más pausada sobre el concepto de espacio local versus espacio mundo, ya que varios errores provinieron de confundir el eje de rotación local del objeto con el eje del mundo. Como sugerencia, incluir un diagrama de árbol jerárquico como referencia visual desde el inicio ahorraría tiempo en la fase de depuración. En conclusión, la implementación del perro articulado consolidó el entendimiento del grafo de escena implícito que se construye con matrices de transformación, habilidad fundamental para cualquier motor de videojuegos o aplicación de simulación 3D.

Bibliografía

- Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). *OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.3* (8th ed.). Addison-Wesley.
- Foley, J. D., van Dam, A., Feiner, S. K., & Hughes, J. F. (1990). *Computer Graphics: Principles and Practice* (2nd ed.). Addison-Wesley.
- de Vries, J. (2020). *LearnOpenGL: Learn Modern OpenGL Graphics Programming in a Step-by-Step Fashion*. Kendall & Welling. Recuperado de <https://learnopengl.com>
- OpenGL Wiki contributors. (2024). *OpenGL Wiki*. Khronos Group. Recuperado de <https://www.khronos.org/opengl/wiki>